理想の人工知能

> **この文書について**
> 本文は、人工知能システムの構造と発展の方向性についての、厳密な検証を目的としないアイデア出しである。以下には筆者の仮説や予想が含まれる。

1. システムの構成

理想の人工知能は、以下の要素で構成される。

1.1. 物理的プロセッサ

CPU、GPU、TPU、FPGA、ASIC、量子コンピュータ、その他の特定計算プロセッサなど。システムの最上位層に位置する。

1.2. インターフェース層

ユーザーに対する入出力を担う。システムのあらゆる層とリンクしていてよい。

1.3. データ層

現実に計算を稼働させるためには、明示的な知識ではないデータ基盤として、コンパイラ、OS、仮想環境、HPC基盤、データベース、プロトコル仕様などが必要である。ただし、ここでは人工知能の機能に着目して抽象化するため、これらを捨象する。本来、これらは以降に述べる内容に依存せず配置できる。

1.3.1. 生データ層

現実世界から直接取り出されるデータ。Web上の知識やローカルの知識などが該当する。システムの最下層に位置する。

1.3.2. キャッシュ層

0個以上存在する。1層につき、生データ層とインターフェース層を1回解釈し、最終的に物理的プロセッサが解釈できる記号列にする。

直感的にイメージされるような単純な直列の積み重ねではなく、DAG（有向非巡回グラフ）的な複雑性を持ちうる。

以下では、この層が今後取りうる厚さと、その構造の適切性について考える。

2. キャッシュ層の要素技術と評価軸

2.1. キャッシュ層の例

現状、人類が発見しているキャッシュ層の例として、以下がある。

- **ルールベースアルゴリズム、あるいは言語モデルに比べて大規模データを必要としない統計的アルゴリズム**
  - ソート・探索・検索
  - フーリエ変換・数値計算などの関数処理
  - 固有値分解などの行列処理

    - 動的計画法・グラフアルゴリズム・離散最適化手法
    - 連続最適化手法・遺伝的アルゴリズム
    - 正規表現・形態素解析・文法的解析
    - ナイーブベイズ・SVM・決定木
    - 形式手法・プログラム解析
- **すでに加工され、相対的に物理的プロセッサにとって解釈しやすくなっているデータ**
    - モデルパラメータ
    - 教師データ・テストデータ
- **LLM**（大規模から小規模まで）
- **入出力を制約することでパフォーマンスを高めた言語モデル**（Jevなど）

2.2. 評価軸

これらの要素には、それぞれ以下の評価軸において優劣がある。

- 計算コスト
- 精度
- 安定性
- 入出力の柔軟性
- 入出力の堅牢性
- 使える場面の広さ
- 機能拡張容易性
- 理解・検証容易性

システム全体として見ると、人類はこれらすべてを同時に満たすことを追求していると言える。

具体的には、まず理解・検証容易性を犠牲にした高度な使い分けによって、それ以外の要素をどうにか実現している。そして、理解・検証容易性を少しずつ改善しながら、その他の要素も改善できないかと努力している。

3. 現状の構造と、考えられる最終形態

3.1. 現状注目されている構造

最近は、（計算コスト、100%の精度と安定性、入出力の堅牢性、理解・検証容易性）を犠牲にすることで、（ある程度の精度と安定性、入出力の柔軟性、使える場面の広さ、機能拡張容易性）を最大化する要素技術として、LLMが発展している。

これにより、以下の構造が注目されている。

```
インターフェース層
      ↓
物理的プロセッサとキャッシュ層（LLMのみとローカルデータベース）
      ↓
生データ層
```

一方で、入出力を制約し、計算コストを圧倒的に上げた言語モデルであるJevが公開され、その用途についての思索が活発化している。

3.2. 個人的に考える最終形態

個人的には、最終形態は以下のようになると考える。

```
計算モジュール的プロセッサ
（LLM / Jev / 前述したアルゴリズムすべて）
    ↓
加工データ層
（DAG的な複雑性を持つ）
    ↓
生データ層

※ インターフェース層は全層とリンクする。
```

ここでいう**加工データ層**は、生データあるいはインターフェース入力を、計算モジュール的プロセッサにとって解釈しやすくしたものである。

外見的には、以下のようなものとして現れると考えられる。

- データベース・ベクトルデータベース
- 計算モジュール的プロセッサに渡す普遍的な命令
- ソースコード
- ナレッジグラフ
- パラメータ列
- 教師データ・テストデータ

3.3. この形態が目指す優位性

この形態の目的は、現状に対して以下の優位性を確保することである。

第一に、目的を達成する知能あたりの計算効率を圧倒的に高めること。

「目的を達成する知能 / 計算コスト」という比率が、現状より圧倒的に大きくなる。計算コストを算出する式を、「必要な機能が寡占されているかどうか」に依存させてもよい。OSSのような形で、各個人のPC程度の計算資源で全機能が達成されるなら、進化が終結するとも言える。

第二に、曖昧な目的を明瞭なタスクの列へ自然に分解すること。

各タスクが本来どの程度の（精度、入出力の堅牢性、機能拡張容易性、理解・検証容易性）を要求するかを、機械が事前知識から最大限推定する。

補えない最小限の部分については人間の判断を要求する。ただし、一度世界のどこかで要求された判断は、リスクがない場合にはOSS的な仕組みで全世界に拡散され、同一の意味を持つ判断が二度要求されることはない。

第三に、機能の本質を人間が認識できるデータとして最大限露出させること。

これによって、ルールベースに近い形で機能拡張や理解・検証を容易にする。文字列や数値を要素とし、それらの間のリンクによってまとまった構造化データになる可能性が高い。

4. 最終形態から逆算した仮説

上記の目的に収束することを前提とすると、現状との差分として、要素単位では次のような想像が妥当になる。

- **LLM / Jevは、今よりかなり「小さい」にもかかわらず、実質的には高機能になるのではないか。** それらは、「外部データを十分に与えられれば、すべての目的について最適な検索・推定ができる、人間共通の知性」のみを有するのではないか。
- **インターフェース層への入力直後に、計算モジュール的プロセッサが最初に呼ぶ、ブートローダー的なデータが存在するのではないか。** それは少なくとも、要求を具体化する命令を含むのではないか。
- **生データ層に新しいデータが入った直後にも、加工操作のために最初に呼ばれるデータが存在するのではないか。**

5. 最初に実装すべき分解

以上の推論を前提として、個人的には、現状の構造に対して最初に以下の分解を導入すべきだと考える。

現状	最初に目指す構造
LLMのみ	LLM / Jev（LLMモデルは変更しない）
インターフェース層	インターフェース層とリンクしたブートローダー

5.1. これを最初に行う利点

- **既存モデルを変更しなくてよい。** 既存のLLM / Jevをまったく変えず、大規模データを作る必要もない。
- **適用できる場面が広い。** 素朴なLLMタスクでは、射程に入る生データの数はゼロか、そこまで多くない。生データを加工する処理は、大規模ソフトウェアの改善などで主に役立つ。一方、インターフェース層への入力は、すべてのLLMタスクに必然的に存在する。したがって、後者のほうが使える場面が広い。
- **後続の作業コストを減らせる。** 最初にこの分解を行えば、後続のタスク（大まかにはプログラミング行為）の労力コストや計算コストもある程度減る。
- **Jevを簡易的に利用できる。** 「ブートローダー中の判定アルゴリズム」や「ブートローダーとしての質の測定」に使え、計算コストが小さい。

6. その知性を何に使うのか

個人的には、最低限、以下の応用があると考える。

- **アプリケーション開発の裾野の拡大。** プログラミング知識を持たない人が、素朴な発想から特定ユースケースのアプリを作れるようになる。その射程が業務用アプリや大規模アプリにまで広がる。
- **エネルギー問題の解決。** ペロブスカイトなどの太陽電池、全固体電池などの二次電池、核融合発電、物流網や発電網の最適化などに使われる。
- **教育の圧倒的な効率化。** 十分な専門知識を得るまでの時間を短縮する。このAIシステムを前提としても、前提としなくてもよい。

7. 既存のWebプログラマーは何をすればよいのか

- **キャッシュ層をリッチにする。** 相対的にこのシステムへ近づくようにキャッシュ層を充実させ、その行為で得られる差分メリット（計算コスト、生産性、供給分散性）を収益に変えていく。現状、知性および計算の恩恵が人類全体に飽和して供給されているわけではまったくない。また、言語の壁は必然的に低くなることを忘れない。
- **レビューや責任を担う。** このシステムに近づいても、レビューをする行為や責任を取る行為は長期的に必要とされる。大量の「元」素人のアプリ生成者に対する顧問業の繁栄も構想できる。

- **周辺分野の専門性を高める。** 教育コストの低下を前提として、応用数学や科学などの専門性を高める。